StudyCoach — A Step-by-Step Build Guide

An AI-assisted, evidence-based study coach you can build in four weeks

Engineering white paper · v1.0

For a 4th-year computer science student

---

1. Overview

1.1 What we are building

StudyCoach is a local-first web application that helps a student answer a deceptively hard question: *which study technique actually works best for me, in this subject?* Instead of sorting students into “learning styles,” StudyCoach treats studying as a small experiment. The student runs evidence-based techniques (active recall, spaced repetition, Feynman / self-explanation, practice questions), records a quick outcome after each session, and the app compares the results over time to surface the winner per subject — with honest uncertainty when the data is thin.

Artificial intelligence appears in exactly three narrow places: generating practice questions from the student’s own material, giving feedback on a free-text answer, and phrasing a plain-language summary of numbers the app has already computed. **AI never decides which technique is best.** That decision is made by a transparent, few-hundred-line recommendation engine that any reader can audit.

This document is a build guide. By the end you will have produced the repository that ships alongside it: a running Next.js app with a seeded demo, a SQLite database, a pluggable LLM layer, and an insights dashboard. It is scoped to be buildable by one motivated 4th-year CS student in about four weeks.

1.2 Why not learning styles

The “learning styles” idea — that each person is a visual, auditory, or kinesthetic learner and should be taught accordingly — is intuitively appealing and empirically unsupported. When researchers test the core prediction (that matching instruction to a person’s supposed style improves learning), the effect does not appear. Meanwhile, a separate body of work identifies techniques that reliably help *most* learners: retrieval practice (the testing effect), distributed practice (spacing), self-explanation, and practice testing consistently outperform re-reading and highlighting.

StudyCoach takes the honest position that follows from this: we do not know in advance which of these high-utility techniques will suit a given student in a given subject, so we should measure rather than label. The onboarding flow forms a *hypothesis*, and the data confirms or overturns it. This framing is the product’s whole reason to exist; keep it central as you build.

1.3 The MVP scope

The MVP is deliberately small and complete:

- **Onboarding** — a few behavioral questions that produce a starting hypothesis.
- **Subjects & material** — the student names what they study and pastes source text.
- **Study sessions** — pick a technique, run a timer, log an outcome check.
- **Question bank** — AI-generated (or hand-written) questions saved per subject, with a

practice + feedback loop.
- **Outcome tracking** — the quick check after each session is the raw evidence.
- **Insights dashboard** — a transparent per-subject comparison of techniques.

Anything beyond this (accounts, multi-user, mobile apps, real spaced-repetition scheduling, imports from PDFs) is a stretch goal in §9, not part of the four-week MVP.

---

2. Architecture

2.1 Stack and rationale

Layer	Choice	Why
Framework	**Next.js 14 (App Router)**	One codebase for UI and API route handlers; no separate backend to deploy for a local app.
Language	**TypeScript**	Types across the data model, API, and UI catch a whole class of bugs for a solo builder.
Styling	**Tailwind CSS**	Fast, consistent styling without a component library to learn.
Database	**SQLite** via **better-sqlite3**	Zero-config, single-file, synchronous, ships prebuilt binaries — ideal local-first.
ORM	**Drizzle ORM** + drizzle-kit	Typed queries and real SQL migrations without heavyweight tooling.
Charts	**Recharts**	Declarative React charts; good enough for bar charts with error bars.
Validation	**Zod**	Validate every API input at the boundary.
AI	**Pluggable provider** (mock / Ollama / OpenAI-compatible)	Runs offline by default; swap in a real model with one env var.

Everything is open source and installable from public registries. The only native module is better-sqlite3, which downloads a prebuilt binary, so there is no compiler step for the common platforms.

2.2 The data model

Seven tables, all owned by a single local user (auth is out of scope for the MVP):

- `users` — one owner row.
- `subjects` — what the student studies (name, color).
- `materials` — pasted source text, per subject.
- `questions` — practice items (type, prompt, answer, `source` = ai|user).

- `sessions` — one study session (subject, technique, minutes, notes, timestamps).
- `outcomes` — the quick check for a session (quiz 0–100, confidence 1–5, recall 0–100).
- `onboarding` — the answers and the computed hypothesis (JSON blobs).

The crucial relationship is `outcomes → sessions → subjects`. A joined row across those three tables is exactly one **evidence record**: *for this subject, using this technique, the outcome was X*. The recommendation engine consumes a stream of these records and nothing else. The full schema lives in `src/lib/db/schema.ts`; a reference copy is in Appendix A.

2.3 The transparent recommendation engine

This is the heart of the app and it is intentionally *not* machine learning. Given the evidence records, the engine (`src/lib/recommend.ts`, built on `src/lib/stats.ts`) does the following:

1. **Score each session.** Blend the three signals into one 0–100 `outcome score`: `0.5 × quiz + 0.3 × recall + 0.2 × confidence%` (confidence 1–5 is first mapped to 0–100). The same weights apply to every technique, which keeps comparisons fair. The weighting is a product decision you can defend and tune; it is written in one place so it is easy to audit.
2. **Aggregate per technique, per subject.** Compute the mean outcome score, a 95% confidence interval (wider when there are fewer sessions), a simple recent-vs-earlier trend, and the total minutes invested.
3. **Rank and classify confidence.** Sort techniques by mean. Declare a **clear signal** only when the leader has at least four sessions and beats the runner-up by more than their confidence intervals overlap. Otherwise the verdict stays **emerging**, and with fewer than three sessions it is **gathering data**. This is what keeps the product honest: it refuses to over-claim on small samples.
4. **Roll up overall.** Across all subjects, report the most-practiced technique and the best-scoring one (used for the home-page “what your data says” verdict).

Because every number is reproducible from the raw rows, a curious student — or a skeptical teacher — can check the app’s reasoning. That auditability is a feature, not an accident.

2.4 The narrow AI boundary

All AI flows through one interface, `LlmProvider` (`src/lib/llm/types.ts`), with three methods: `generateQuestions`, `feedback`, and `summarizeInsights`. Three implementations exist:

- **`mock`** — deterministic heuristics, fully offline, the default. It splits material into sentences and builds cloze deletions, recall prompts, and self-explanation prompts. It is good enough that the whole app works with no model at all.
- **`ollama`** — talks to a local Ollama server (open-source models).
- **`openai`** — any OpenAI-compatible `/chat/completions` endpoint (OpenAI, LM Studio, llama.cpp, vLLM).

A `withFallback` helper wraps every call: if the configured model throws, it silently falls back to the mock and reports which provider actually answered, so the UI can label AI output honestly and the demo never breaks. Keeping AI behind this seam is what lets the core product stay transparent and data-based while still benefiting from generation where generation genuinely helps.

2.5 Request lifecycle

A page such as the dashboard is a client component. On mount it calls the JSON API (`/api/insights`, `/api/onboarding`, …). Each route handler validates input with Zod, reads or writes SQLite through Drizzle, runs pure domain logic (`recommend.ts`, `hypothesis.ts`), and returns JSON. There is no hidden state: the SQLite file *is* the state, and it is human-readable.

---

3. Before you start (Day 0)

Prerequisites: Node.js 18.17+ (20 or 22 recommended), Git, and a code editor. No database server, no cloud account, no API key.

Confirm your toolchain:

```
node --version    # v18.17+ (v20/v22 ideal)
npm --version
```

Decide your working rhythm. The plan below is four one-week sprints. Each sprint ends with a runnable app and a short **acceptance check** you can demo. Do not move on until the check passes — a working slice each week is worth more than a half-built everything.

---

4. Week 1 — Foundations, data model, and the first vertical slice

Goal: a Next.js app that boots, a migrated SQLite database, and one feature working end-to-end (subjects) so you have proven the whole stack talks to itself.

4.1 Scaffold the project

Create a Next.js + TypeScript + Tailwind app and add the dependencies:

```
npx create-next-app@14 studycoach --ts --tailwind --app --src-dir --eslint --import-alias
"@/*"
cd studycoach
npm install better-sqlite3 drizzle-orm zod recharts
npm install -D drizzle-kit @types/better-sqlite3 tsx
```

If you prefer to learn from a finished reference, the accompanying repository already has this scaffold plus the design tokens in `tailwind.config.ts` and `src/app/globals.css`. Read those two files early — they define the visual language (see §8).

4.2 Define the schema and connection

Write `src/lib/db/schema.ts` with the seven tables from §2.2 using Drizzle's `sqliteTable`. Store timestamps as ISO-8601 text so the database file stays readable. Then write `src/lib/db/index.ts`, which opens a single cached better-sqlite3 connection (cache it on `globalThis` so Next's hot reload does not open the file repeatedly) and enables `journal_mode = WAL` and `foreign_keys = ON`.

Add a `drizzle.config.ts` pointing at the schema, and two npm scripts:

```
// package.json → "scripts"
"db:generate": "drizzle-kit generate",
"db:migrate":  "tsx scripts/migrate.ts",
"db:seed":     "tsx scripts/seed.ts",
"db:reset":    "rm -f db/studycoach.sqlite && npm run db:migrate && npm run db:seed",
"setup":       "npm run db:migrate && npm run db:seed"
```

`scripts/migrate.ts` opens the database and calls Drizzle's `migrate(...)` against the `./drizzle` folder. Generate the first migration and apply it:

```
npm run db:generate     # writes drizzle/0000_*.sql from the schema
npm run db:migrate      # applies it to db/studycoach.sqlite
```

Common pitfall: if you later put async work in a script, wrap it in an `async function main()` and call it — top-level `await` breaks under the CommonJS transform tsx uses by default.

4.3 A single-user helper

Write `src/lib/user.ts` with `getCurrentUser()` that returns (creating if needed) the one owner row with id `local-user`. Every API route resolves ownership through this function. When you later add real auth, this is the only place that changes.

4.4 First vertical slice: subjects

Build the thinnest complete feature to prove the stack:

1. **API** — `src/app/api/subjects/route.ts` with `GET` (list) and `POST` (create, validated by Zod). Add `export const dynamic = "force-dynamic"` and `export const runtime = "nodejs"` so GETs always hit the database and native SQLite runs on the Node runtime.
2. **Page** — `src/app/subjects/page.tsx`, a client component that fetches the list and posts a new subject (name + color).
3. **Shell** — a `src/app/layout.tsx` with a sidebar nav and a small fetch helper `src/lib/client.ts` (`getJSON` , `postJSON`).

Acceptance check (end of Week 1): `npm run dev`, open `/subjects`, create a subject, reload, and see it persist. You have now exercised UI → API → Zod → Drizzle → SQLite and back.

---

5. Week 2 — Onboarding, sessions, and outcome tracking

Goal: the student can complete onboarding and log study sessions with outcome checks. This produces the raw evidence the whole app depends on.

5.1 The techniques catalog

Create `src/lib/techniques.ts`: a typed list of the five techniques (four active plus a

re-reading *control*), each with a label, a one-line blurb, a short how-to, and a pointer to the evidence (see §10). Marking re-reading explicitly as the control is a nice honest touch — it gives every other technique a baseline to beat.

5.2 Onboarding → starting hypothesis

Onboarding must not feel like a personality quiz. Ask behavioral questions: *"You re-read your notes and feel like you know it — a few days later, how much sticks?"*, *"What trips you up most?"*, *"How consistent is your schedule?"*. Put the questions and a transparent scoring table in `src/lib/hypothesis.ts`. Each answer adds weight to one or more techniques; `computeHypothesis` sums them, ranks the techniques, and writes a one-paragraph rationale that explicitly frames the result as a guess to be tested.

Wire up:

- `POST /api/onboarding` — accepts the answers, calls `computeHypothesis`, stores both as JSON.
- `GET /api/onboarding` — returns the latest hypothesis (or `{completed:false}`).
- `src/app/onboarding/page.tsx` — a stepper that shows one question at a time, then the result.

Keep the scoring rules readable. The point of the product is that a student could open this file and understand exactly why they got the hypothesis they did.

5.3 Sessions and the outcome check

The study page (`src/app/study/page.tsx`) is the core interaction:

1. Pick a subject, then a technique (show a *suggested* technique pulled from either the current best data for that subject or the onboarding hypothesis).
2. Optionally run a simple count-up **focus timer** to fill the minutes.
3. Take the **outcome check**: a quiz/self-test slider (0–100), a confidence segment (1–5), and an unaided-recall slider (0–100). Show the computed outcome score live so the student sees how the pieces combine.

On submit, `POST /api/sessions` writes the session and, in the same call, its outcome (accept an optional inline `outcome` object). Keep a standalone `POST /api/outcomes` too, for logging a later review of the same material. Validate everything with Zod and clamp ranges.

Acceptance check (end of Week 2): complete onboarding, then log several sessions across different techniques and subjects. Inspect `db/studycoach.sqlite` (any SQLite viewer) and confirm `sessions` and `outcomes` rows are being written correctly.

---

6. Week 3 — The question bank and the narrow AI layer

Goal: turn a student's material into practice, and add the answer-feedback loop — all behind a swappable AI seam that works offline.

6.1 The provider interface

Define `LlmProvider` in `src/lib/llm/types.ts` with `generateQuestions`, `feedback`, and `summarizeInsights`. Implement three providers:

- `mock.ts` — the offline heuristic generator and a token-overlap feedback function. Build this **first**; it is your development default and your safety net.
- `ollama.ts` — POST to `${OLLAMA_BASE_URL}/api/chat` with `format:"json"` for question generation.
- `openai.ts` — POST to `${OPENAI_BASE_URL}/chat/completions` with a JSON response format.

Add `index.ts` with `getProvider()` (reads `LLM_PROVIDER`) and `withFallback()` (runs the real provider, catches failures, falls back to mock, and reports which one answered). Put your prompts in the providers; keep them strict — *"return only JSON; questions must be answerable strictly from the provided material."*

6.2 Generation and storage

- `POST /api/materials` — save pasted text under a subject.
- `POST /api/questions/generate` — load the material, call `withFallback(p => p.generateQuestions(...))`, persist the results with `source:"ai"`, and return them along with the provider used and whether it fell back.
- `GET /api/questions` / `POST /api/questions` — list and hand-add questions.

The question bank page (`src/app/questions/page.tsx`) lets the student paste notes, choose how many questions, generate, filter by type, reveal answers, and **practice**: type an answer, call `POST /api/feedback`, and show the model's comment. Label AI output clearly and surface the fallback state ("model unavailable — used the built-in generator").

Acceptance check (end of Week 3): paste a paragraph, generate questions offline (mock), practice one, and get feedback. Then, if you have Ollama, set `LLM_PROVIDER=ollama` and confirm the same flows use the real model — and still degrade gracefully if you stop the server.

6.3 A note on prompt-injection and trust

Material is untrusted text. Never let it silently change app behavior: the LLM only *reads* material to produce questions; it has no tools and no write access. Keep it that way. If you later add file import, sanitize and size-limit input before it reaches a prompt.

---

7. Week 4 — Insights, polish, seed data, and hardening

Goal: the payoff — a transparent insights dashboard — plus the demo seed and a final polish pass.

7.1 Statistics and the engine

Write `src/lib/stats.ts` (mean, standard deviation, standard error, a z-based 95% CI, the `outcomeScore` blend, and a recent-vs-earlier `trend`). Then write `src/lib/recommend.ts` exactly as described in §2.3: group evidence by subject then technique, compute `TechniqueStat`s,

classify confidence, generate a plain-language headline in code, and roll up overall totals. Keep it pure (no I/O) so it is trivial to unit-test.

7.2 The insights API and dashboard

`GET /api/insights` joins `outcomes → sessions → subjects`, runs `computeInsights`, then calls `withFallback(p => p.summarizeInsights(...))` on the *already-computed* numbers. Return the report, the summary, and the provider. The dashboard (`src/app/insights/page.tsx`) renders, per subject: a confidence badge, the code-generated headline, and a horizontal Recharts bar chart of mean outcome score per technique with 95% CI whiskers and the leader highlighted. Add a collapsible "how this is computed" panel — transparency is part of the UX, not an afterthought.

On the home page, make the **hypothesis-vs-evidence** contrast the hero: the onboarding guess on one side, "what your data says" on the other. That single comparison is the product's thesis.

7.3 Seed a lively demo

Write `scripts/seed.ts` that wipes and repopulates: one user, three subjects, an onboarding row, a small AI-generated (mock) question bank, and ~40–50 sessions whose outcome means are designed so that two subjects show a **clear** winner and one stays **emerging**. Use low within-technique variance and a real gap between means to produce clear signals; narrow the gap to demonstrate the honest "still close" state. Spread timestamps over a few weeks so trends are meaningful. A good demo that opens already telling a story is worth the effort.

7.4 Hardening checklist

- Type-check the whole project: `npm run typecheck` (aim for zero errors).
- Run a full `npm run build` — it type-checks, lints, and exercises the static/dynamic split, catching problems `npm run dev` hides (see §9.1).
- Validate every API input with Zod and clamp numeric ranges.
- Handle empty states everywhere (no subjects, no sessions, no insights yet).
- Respect `prefers-reduced-motion`; ensure visible keyboard focus; check mobile layout.
- Confirm `npm run db:reset && npm run dev` gives a clean, populated app from scratch.

Final acceptance check: a new clone runs with `npm install && npm run setup && npm run dev`, shows a populated dashboard, and every feature works offline with no API key.

---

8. Design language

The app avoids the generic "AI starter" look. Its identity is a **study-lab notebook**: a cool paper background, ink-near-black text, a single restrained indigo accent, monospace for data and labels, and a faint graph-paper texture behind the hero and insight surfaces. The confidence colors are *semantic* — green for a clear signal, amber for emerging, grey for gathering data — so color encodes meaning rather than decoration. The signature element is the hypothesis-vs-evidence panel on the home page. All of this lives in `tailwind.config.ts` and `src/app/globals.css`; change the tokens there and the whole app follows. Spend your visual boldness in one place (the hero) and keep everything else quiet.

9. Testing, validation, and stretch goals

9.1 Testing and verification

You do not need a heavy test harness to be confident this app works — you need a few cheap layers, applied in order from fastest to slowest. This is the exact sequence used to verify the reference build, and every step caught something worth catching at least once.

Layer 1 — static checks (seconds). Run `npm run typecheck` and, importantly, a full `npm run build`. A production build is an underrated test: it type-checks, lints, and statically analyses every page, and it exercises the App Router's static/dynamic split — so it surfaces problems (an accidental server-only import in a client component, a route that should be dynamic but isn't) that `npm run dev` will happily hide. Confirm your API routes show as dynamic (`ƒ`) and your pages as static (`○`) in the build output.

Layer 2 — database reproducibility (seconds). Delete the database and run `npm run db:reset`. Migrations plus seed must reproduce a clean, populated database every time with zero manual steps. Then assert the shape you expect — for the seed, every session should have exactly one outcome (no orphans). If a teammate cannot get a working database from a fresh clone in one command, the onboarding story is broken.

Layer 3 — runtime smoke tests (a minute). Start the server and, with `curl`, hit every page (expect `200`), every read API (assert the counts and the narrative — three subjects, two "clear" and one "emerging"), and every write flow (create a subject, log a session with an inline outcome, generate questions, request feedback). Then close the loop: confirm the new data actually shows up in the insights report and the question bank. Finally, send deliberately bad input and assert it is rejected with a `422` — validation you never test is validation you do not have.

Layer 4 — unit tests on the pure modules (the highest-value tests). `stats.ts`, `recommend.ts`, and `hypothesis.ts` are pure functions with no I/O, which makes them a joy to test. A lightweight runner such as `vitest` is plenty. The recommendation engine is where the product's integrity lives, so test it at the boundaries by feeding it synthetic evidence:

- Fewer than three sessions on the leader → confidence is *gathering data*, never a winner.
- A wide, well-separated gap with enough sessions → *clear*, and the right technique wins.
- A small gap → *emerging*, not *clear* (the app must refuse to over-claim).
- A clear leader whose runner-up has too few sessions → still not *clear* yet.
- All-maximum inputs → an outcome score of exactly 100 (the blend is sane).

A cautionary tale worth internalising. When first writing the small-gap case above, it is tempting to give every session in a technique the *same* score (say, five sessions all at 79 versus five all at 77) and expect "emerging." It will instead come back "clear" — and that is not a bug, it is the statistics being honest. With zero variance the confidence intervals collapse to zero width, so *any* non-zero gap is perfectly separable. Real study data always has spread. The lesson: **your synthetic test data must carry realistic variance**, because the confidence machinery you are testing is precisely a machine for reasoning about variance. Add a little jitter to each score and the small gap correctly reads "emerging" again. Encoding that one insight as a test is a better lesson in what the engine does than a page of prose.

This ships as runnable tests. The reference repository encodes exactly these cases as a Vitest suite under `src/lib/__tests__/` (`stats.test.ts`, `recommend.test.ts`, `hypothesis.test.ts`) — including the zero-variance trap and its jittered counterpart, side by side. Run them with `npm test`. A useful discipline once they are green: temporarily break the "clear signal" rule in `recommend.ts` and confirm the two "emerging" boundary tests go red. A suite that never fails when you introduce a bug is not testing anything; that quick mutation check is how you earn trust in it.

What these layers do not cover. The `ollama` and `openai` providers need a live model server to exercise directly, so automated tests generally run against the offline `mock`. That is an acceptable trade because of the fallback seam (§2.4): when a real model is unavailable the app degrades to the mock path, which *is* covered — so the failure mode you most care about is the one you test by default. If you want real-provider coverage, gate those tests behind an env flag and run them only when a local model is up.

9.2 Validating the concept

The product makes an empirical claim, so evaluate it empirically. Two directions:

- **Self-experiment:** log real sessions for two weeks and check whether the recommendation is stable and matches your felt experience.
- **Formative study:** have a handful of students use it and interview them. Does the hypothesis-vs-evidence framing change how they study? Do they trust a "clear signal"?

9.3 Stretch goals (post-MVP)

- Real spaced-repetition scheduling (SM-2/FSRS) that tells the student *when* to review.
- Import material from PDFs or a URL (with sanitization).
- Per-question difficulty tracking and adaptive practice.
- Accounts and sync (introduce auth at `src/lib/user.ts`).
- Export a study report; richer trend charts over time.
- A proper statistical test (e.g., Welch's t-test) behind the "clear signal" label.

---

10. Ethics, privacy, and the evidence base

Privacy. Everything is local: one SQLite file on the student's machine, no telemetry, no account. If you add a hosted model, be explicit that pasted material leaves the device; prefer a local model (Ollama) for privacy-sensitive material.

Honesty. The app must never over-claim. The confidence tiers exist precisely so it says "gathering data" instead of inventing a winner from three sessions. Preserve that discipline in any change you make.

Evidence base (starting points for your own reading). Roediger & Karpicke (2006) on the testing effect; Cepeda et al. (2006) on distributed practice; Chi et al. (1994) on self-explanation; Dunlosky et al. (2013), *Improving Students' Learning With Effective Learning Techniques*, for a comparative review that rates practice testing and distributed practice highest and re-reading/highlighting low; and Pashler et al. (2008) for the critical review of learning styles. These inform the technique catalog and the product's core stance.

Appendix A — Data model reference

```
users(id, name, created_at)
subjects(id, user_id→users, name, color, created_at)
materials(id, user_id→users, subject_id→subjects, title, content, created_at)
questions(id, user_id→users, subject_id→subjects, material_id→materials?,
          type[recall|practice|feynman|cloze], prompt, answer, source[ai|user], created_at)
sessions(id, user_id→users, subject_id→subjects, technique, material_id→materials?,
         planned_minutes, actual_minutes, notes, started_at, ended_at?)
outcomes(id, session_id→sessions, quiz_score[0..100], confidence[1..5], recall[0..100],
         notes, created_at)
onboarding(id, user_id→users, answers(json), hypothesis(json), created_at)
```

Appendix B — API reference

Method & path	Purpose
`GET /api/subjects`	List subjects
`POST /api/subjects`	Create a subject
`GET /api/materials?subjectId=`	List materials
`POST /api/materials`	Save pasted material
`GET /api/questions?subjectId=`	List questions
`POST /api/questions`	Add a question by hand
`POST /api/questions/generate`	AI-generate questions from material
`GET /api/sessions`	List sessions
`POST /api/sessions`	Log a session (optional inline outcome)
`POST /api/outcomes`	Log an outcome for an existing session
`GET /api/onboarding`	Latest hypothesis
`POST /api/onboarding`	Save answers, compute hypothesis
`POST /api/feedback`	Feedback on a free-text answer
`GET /api/insights`	Computed report + AI summary

Appendix C — Environment variables

Variable	Default	Meaning
`LLM_PROVIDER`	`mock`	`mock` | `ollama` | `openai`
`OLLAMA_BASE_URL`	`http://localhost:11434`	Ollama server
`OLLAMA_MODEL`	`llama3.1`	Ollama model tag
`OPENAI_BASE_URL`	`https://api.openai.com/v1`	OpenAI-compatible endpoint
`OPENAI_API_KEY`	*(empty)*	Key for the endpoint (if required)

`OPENAI_MODEL`	`gpt-4o-mini`	Model name
`DATABASE_PATH`	`./db/studycoach.sqlite`	SQLite file path

Appendix D — Glossary

- **Outcome score** — the 0–100 blend of quiz, recall, and confidence for one session.
- **Evidence record** — one joined `outcome + session + subject` row.
- **Clear signal** / **Emerging** / **Gathering data** — the three confidence tiers the engine reports.
- **Starting hypothesis** — the onboarding-derived first guess, to be tested by data.
- **Control** — re-reading, included as an honest baseline for the active techniques to beat.